



JERUSALEM COLLEGE OF ENGINEERING

AN AUTONOMOUS INSTITUTION

Approved by AICTE, New Delhi, Affiliated to Anna University Chennai,
Accredited by NBA, New Delhi and Accredited by NAAC with "A" Grade.
Velachery Main Road, Narayanapuram, Pallikaranai, Chennai, Tamil Nadu - 600100.

UNIT II

DATABASE DESIGN

Entity-Relationship model – E-R Diagrams – Enhanced-ER Model – ER-to-Relational Mapping
Functional Dependencies–Non-loss Decomposition–First-Second-Third Normal Forms Dependency
Preservation – Boyce/Codd Normal Form – Multi-valued Dependencies and Fourth Normal Form–
Join Dependencies and Fifth Normal Form.

2.1 ENTITY-RELATIONSHIP MODEL

Methodology:

1. Use E-R model to get a high-level graphical view of essential components of enterprise and how they are related
2. Then convert E-R diagram to SQL DDL, or whatever database model you are using

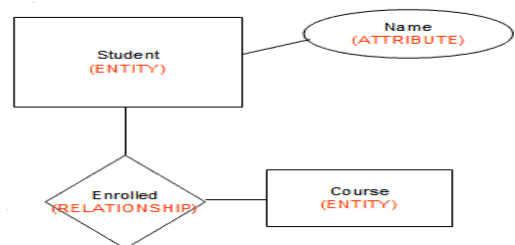
E-R Model is not SQL based. It's not limited to any particular DBMS. It is a conceptual and semantic model – captures meanings rather than an actual implementation

The E-R Model: The enterprise is viewed as set of

- Entities
- Relationships among entities

Symbols used in E-R Diagram

- Entity – rectangle
- Attribute – oval
- Relationship – diamond
- Link - line



Entities and Attributes

Entity: an object that is involved in the enterprise and that be distinguished from other objects. (not shown in the ER diagram--is an instance)

- Can be person, place, event, object, concept in the real world
- Can be physical object or abstraction
- Ex: "John", "CSE305"

Entity Type: set of similar objects or a category of entities; they are well defined

- A rectangle represents an entity set
- Ex: *students*, *courses*
- We often just say "entity" and mean "entity type"

Attribute: describes one aspect of an entity type; usually [and best when] single valued and indivisible (atomic)

- Represented by oval on E-R diagram
- Ex: *name*, *maximum enrollment*
- May be multi-valued – use double oval on E-R diagram
- May be composite – attribute has further structure; also use oval for composite attribute, with ovals for components connected to it by lines
- May be derived – a virtual attribute, one that is computable from existing data in the database, use dashed oval. This helps reduce redundancy

Entity Types

An entity type is named and is described by set of attributes

- Student: Id, Name, Address, Hobbies

Domain: possible values of an attribute.

- Note that the value for an attribute can be a set or list of values, sometimes called "multi-valued" attributes
- This is in contrast to the pure relational model which requires atomic values
- E.g., (111111, John, 123 Main St, (stamps, coins))

Key: subset of attributes that uniquely identifies an entity (candidate key)

Entity Schema:

The meta-information of entity type name, attributes (and associated domain), key constraints

Entity Types tend to correspond to nouns; *attributes* are also nouns albeit descriptions of the parts of entities

May have null values for some entity attribute instances – no mapping to domain for those instances

Keys

- Superkey: an attribute or set of attributes that uniquely identifies an entity--there can be many of these
- Composite key: a key requiring more than one attribute
- Candidate key: a superkey such that no proper subset of its attributes is also a superkey (minimal superkey – has no unnecessary attributes)
- Primary key: the candidate key chosen to be used for identifying entities and accessing records. Unless otherwise noted "key" means "primary key"
- Alternate key: a candidate key not used for primary key
- Secondary key: attribute or set of attributes commonly used for accessing records, but not necessarily unique
- Foreign key: term used in relational databases (but not in the E-R model) for an attribute that is the primary key of another table and is used to establish a relationship with that table where it appears as an attribute also.

So a foreign key value occurs in the table and again in the other table. This conflicts with the idea that

a value is stored only once; the idea that a fact is stored once is not undermined.

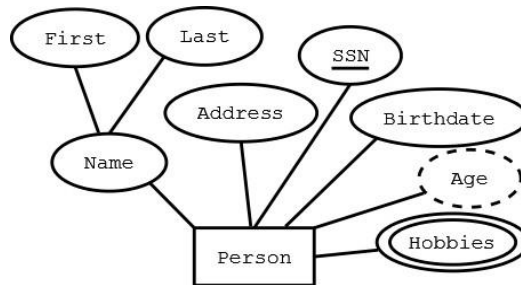
2.2. GRAPHICAL REPRESENTATION IN E-R DIAGRAM

Rectangle -- Entity

Ellipses -- Attribute (underlined attributes are [part of] the primary key) Double ellipses -- multi-valued attribute

Dashed ellipses-- derived attribute, e.g. age is derivable from birthdate and current date.

[Drawing notes: keep all attributes above the entity. Lines have no arrows. Use straight lines only]



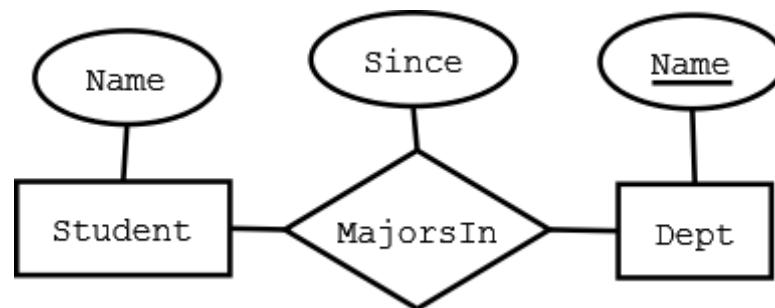
Relationships

Relationship: connects two or more entities into an association/relationship

- "John" majors in "Computer Science"

Relationship Type: set of similar relationships

- *Student* (entity type) is related to *Department* (entity type) by *MajorsIn* (relationship type).



Relationship Types may also have attributes in the E-R model. When they are mapped to the relational model, the attributes become part of the relation. Represented by a diamond on E-R diagram.

Relationship types can have descriptive attributes like entity sets

Relationships tend to be verbs or verb phrases; attributes of relationships are again nouns

[Drawing tips: relationship diamonds should connect off the left and right points; Dia can label those points with cardinality; use Manhattan connecting line (horizontal/vertical zigzag)]

Attributes and Roles

An attribute of a relationship type adds additional information to the relationship

- e.g., "John" majors in "CS" *since* 2000

- John and CS are related
- 2000 describes the relationship - it's the value of the *since* attribute of *MajorsIn* relationship type
- time stamps of updates or establishment of a relationship between two entities can be attributed here rather than with the entities.

The role of a relationship type names one of the related entities. The name of the entity is usually the role name. e.g., "John" is value of *Student* role, "CS" value of *Department* role of *MajorsIn* relationship type (John, CS, 2000) describes a relationship

Situation: relationships can relate elements of same entity type

e.g., ReportsTo relationship type relates two elements of Employee entity type:

- Bob reports to Mary since 2000

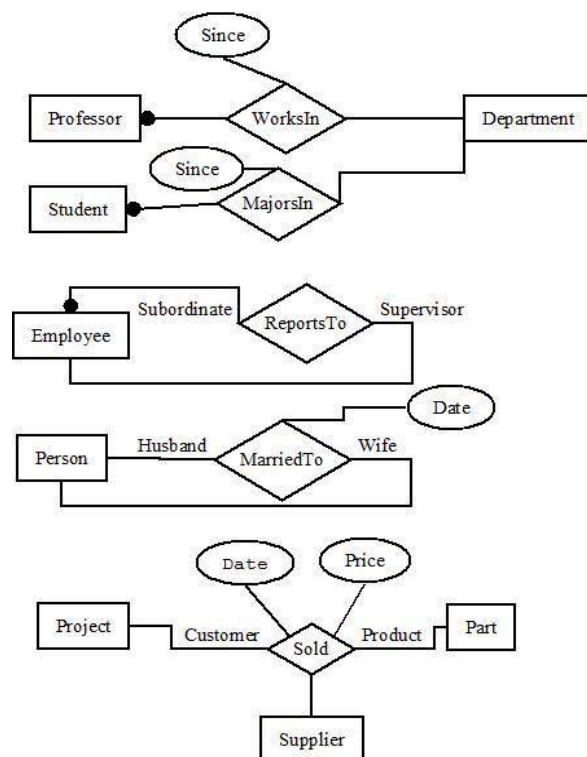
We do not have distinct names for the roles. It is not clear who reports to whom. **Solution:** the role name of relationship type need not be same as name of entity type from which participants are drawn

- *ReportsTo* has roles *Subordinate* and *Supervisor* and attribute *Since*
- Values of *Subordinate* and *Supervisor* both drawn from entity type *Employee*

It is optional to name role of each entity-relationship, but helpful in cases of

- Recursive relationship – entity set relates to itself
- Multiple relationships between same entity sets

Roles are edges labeled with role names (omitted if role name = name of entity set). Most attributes have been omitted.



Relationship Type

Relationship types are described by the set of roles (entities) and [optional] attributes

- e.g., MajorsIn: Student, Department, Since

Think that entities are nouns; relationship types are often verbs

- students and departments are the entities (nouns) and roles in relationship types
- majors is the relationship type (verb)
- i.e., "student" "majors in" "department"

Here we have equate the role name (Student) the name of the entity type (Student) of the participant in the relationship.

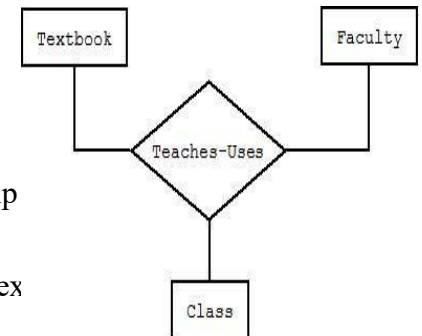
Degree of relationship

The number of roles in the relationship

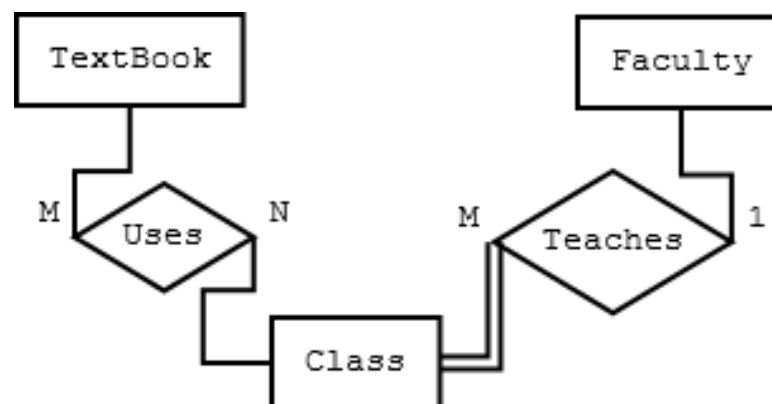
Binary – links two entity sets; set of ordered pairs (most common)

Ternary – links three entity sets; ordered triples (rare). If a relationship exists among the three entities, all three must be present

N-ary – links n entity sets; ordered n-tuples (very rare). If a relationship exists all must be present. Cannot represent subsets.

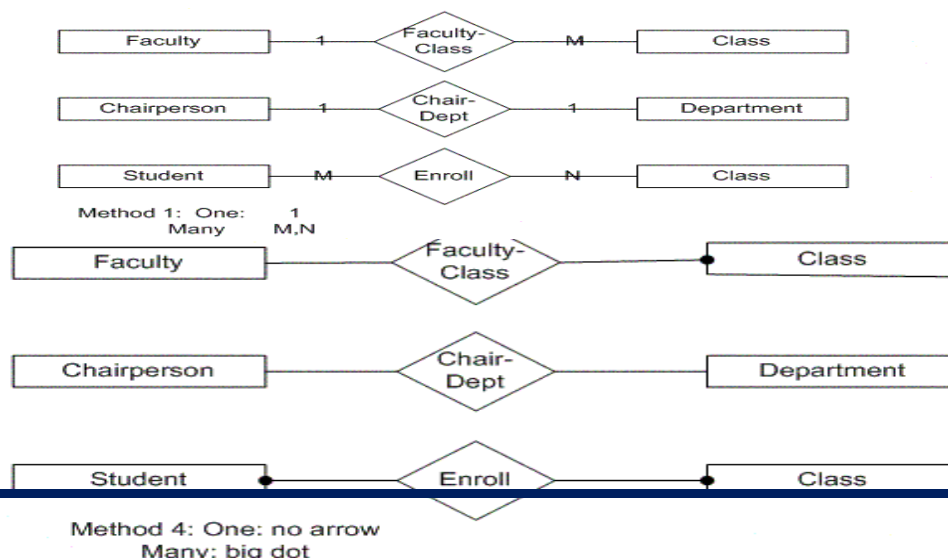


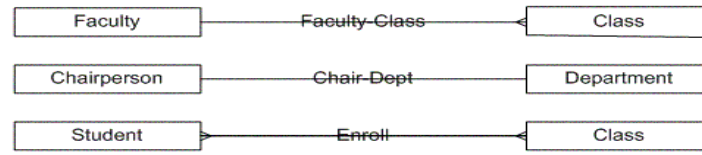
Note: ternary relationships may sometimes be replaced by two binary relationships (see book Figures). Semantic equivalence between ternary relationships and two binary ones are not necessarily true.



Cardinality of Relationships

Cardinality is the number of entity instances to which another entity set can map under the relationship. This does not reflect a requirement that an entity has to participate in a relationship.





Method 5: One: no arrow
Many: crow's feet

Participation is another concept.

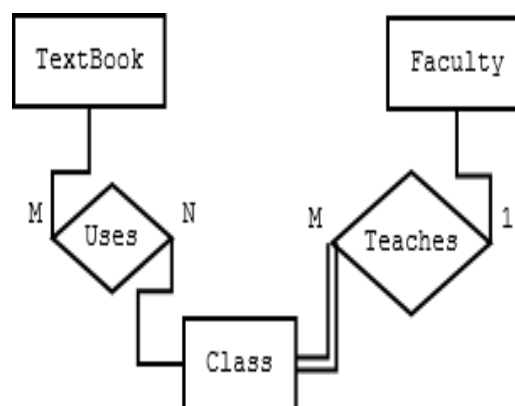
- One-to-one: X-Y is 1:1 when each entity in X is associated with at most one entity in Y, and each entity in Y is associated with at most one entity in X.
- One-to-many: X-Y is 1:M when each entity in X can be associated with many entities in Y, but each entity in Y is associated with at most one entity in X.

Many-to-many: X:Y is M:M if each entity in X can be associated with many entities in Y, and each entity in Y is associated with many entities in X ("many" => one or more and sometimes zero)

Relationship Participation Constraints

Total participation

- Every member of entity set must participate in the relationship



- Represented by *double line* from entity rectangle to relationship diamond
- E.g., A *Class* entity cannot exist unless related to a *Faculty* member entity in this example, not necessarily at Juniata.
- You can set this double line in Dia
- In a relational model we will use the *references* clause.

Key constraint

- If every entity participates in *exactly* one relationship, both a total participation and a key constraint hold
- E.g., if a class is taught by only one faculty member.

Partial participation

- Not every entity instance must participate

- Represented by single line from entity rectangle to relationship diamond
- E.g., A *Textbook* entity can exist without being related to a *Class* or vice versa.

Existence Dependency and Weak Entities

Existence dependency: Entity Y is existence dependent on entity X is each instance of Y must have a corresponding instance of X

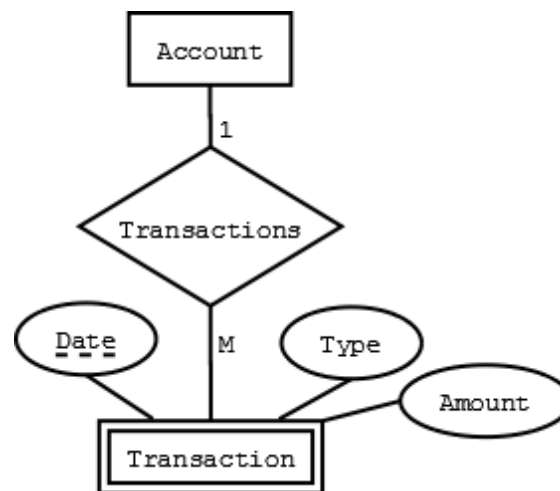
In that case, Y must have total participation in its relationship with X

If Y does not have its own candidate key, Y is called a weak entity, and X is strong entity

Weak entity may have a partial key, called a discriminator, that distinguishes instances of the weak entity that are related to the same strong entity

Use double rectangle for weak entity, with double diamond for relationship connecting it to its associated strong entity

Note: not all existence dependent entities are weak – the lack of a key is essential to definition



Schema of a Relationship Type

Contains the following features:

Role names, R_i , and their corresponding entity sets. Roles must be single valued (the number of roles is called its degree)

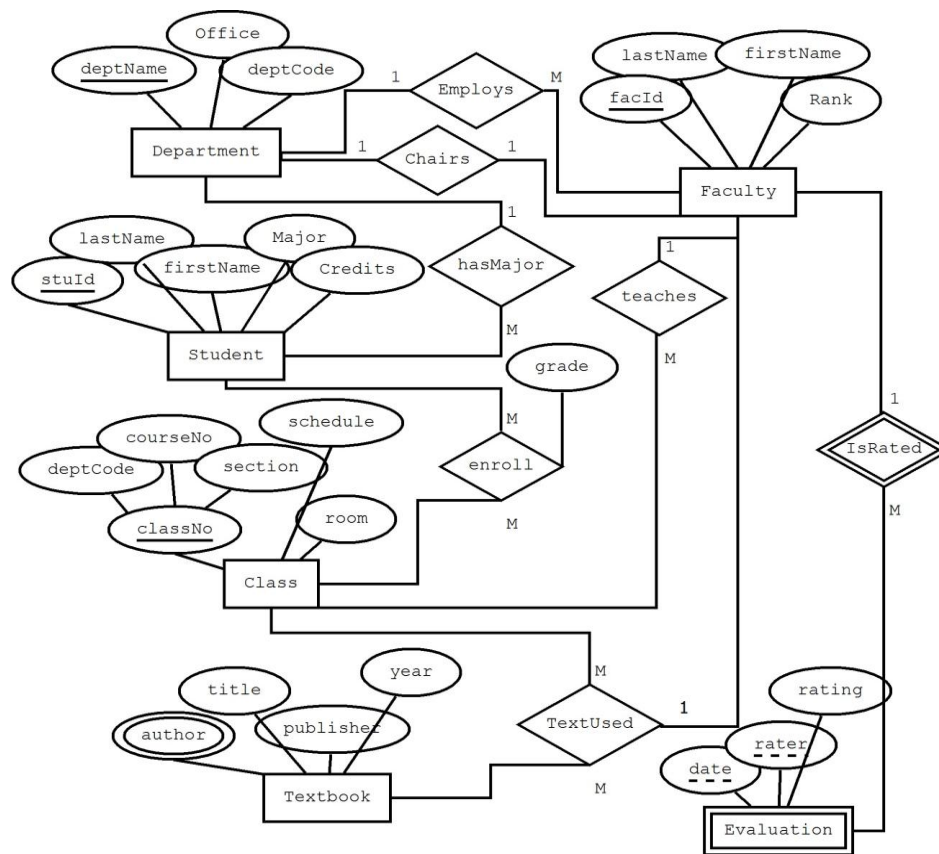
Attribute names, A_j , and their corresponding domains. Attributes in the E-R model may be set or multi-valued.

Key: Minimum set of roles and attributes that uniquely identify a relationship

Relationship: $\langle e_1, \dots, e_n; a_1, \dots, a_k \rangle$

- e_i is an entity, a value from R_i 's entity set
- a_j is a set of attribute values with elements from domain of A_j

ER Diagram Example



This was produced with Dia. It is the same as the figure in the book using instructor's preferred style.

2.3 ENHANCED ENTITY-RELATIONSHIP MODEL

The Extended Entity-Relationship Model is a more complex and high-level model that extends an E-R diagram to include more types of abstraction, and to more clearly express constraints.

All of the concepts contained within an E-R diagram are included in the EE-R model, along with additional concepts that cover more semantic information. These additional concepts include generalization/specialization, union, inheritance, and subclass/superclass.

- Specialization
- Generalization
- Attribute Inheritance
- Constraints on Generalizations
- Aggregation
- Reduction to Relation Schemas

Specialization

An entity set may include subgroupings of entities that are distinct in some way from other entities in the set. The E-R model provides a means for representing these distinctive entity groupings. As an example, the entity set *person* may be further classified as one of the following:

- *employee*.
- *student*.

The process of designating subgroupings within an entity set is called **specialization**.

Generalization

The refinement from an initial entity set into successive levels of entity subgroupings represents a **top-down** design process in which distinctions are made explicit.

Higher- and lower-level entity sets also may be designated by the terms **superclass** and **subclass**, respectively.

Attribute Inheritance

A crucial property of the higher- and lower-level entities created by specialization and generalization is **attribute inheritance**. The attributes of the higher- level entity sets are said to be **inherited** by the lower-level entity sets.

For example, *student* and *employee* inherit the attributes of *person*. Thus, *student* is described by its *ID*, *name*, and *address* attributes, and additionally a *tot cred* attribute; *employee* is described by its *ID*, *name*, and *address* attributes, and additionally a *salary* attribute.

A higher-level entity set with attributes and relationships that apply to all of its lower-level entity sets.

Lower-level entity sets with distinctive features that apply only within a particular lower-level entity set.

Constraints on Generalizations

Condition-defined

In condition-defined lower-level entity sets, membership is evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate.

User-defined

User-defined lower-level entity sets are not constrained by a membership condition; rather, the database user assigns entities to a given entity set.

Disjoint

A *disjointness constraint* requires that an entity belong to no more than one lower-level entity set.

Overlapping

In *overlapping generalizations*, the same entity may belong to more than one lower-level entity set within a single generalization.

Aggregation

- One limitation of the E-R model is that it cannot express relationships among relationships.
- The best way to model a situation such as the one just described is to use aggregation.
- **Aggregation** is an abstraction through which relationships are treated as higher- level entities.

2.4 FUNCTIONAL DEPENDENCIES

The whole database is described by a single **universal** relation schema $R = \{A_1, 2, \dots, A_n\}$.

Definition:

- A **functional dependency**, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a **constraint** on the possible tuples that can form a relation state r of R .
- The constraint is that, for any two tuples t_1 and t_2 in r that have $t_1[X] = t_2[X]$, they must also have $t_1[Y] = t_2[Y]$.
- The values of the Y component of a tuple in r depend on, or are *determined by*, the values of the X component.
- The values of the X component of a tuple uniquely (or **functionally**) *determine* the values of the Y component.
- There is a functional dependency (FD or f.d) from X to Y , or that Y is **functionally dependent** on X .
- X functionally determines Y in a relation schema R if, and only if, whenever two tuples of $r(R)$ agree on their X -value, they must necessarily agree on their Y value.

Note the following:

- If a constraint on R states that there cannot be more than one tuple with a given X -value in any relation instance $r(R)$
- That is, X is a **candidate key** of R —this implies that $X \rightarrow Y$ for any subset of attributes Y of R .
- If $X \rightarrow Y$ in R , this does not say whether or not $Y \rightarrow X$ in R .
- A functional dependency is a property of the **semantics** or **meaning of the attributes**.
- Whenever the semantics of two sets of attributes in R indicate that a functional dependency should hold, specify the dependency as a constraint.
- Relation extensions $r(R)$ that satisfy the functional dependency constraints are called **legal relation states** (or **legal extensions**) of R .

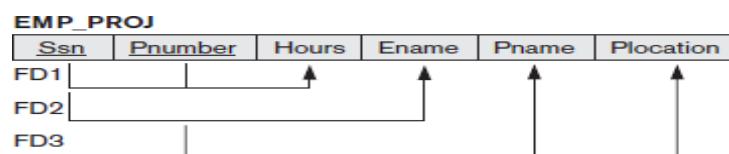


Fig 2.7. Relation schemas EMP_PROJ.

Consider the relation schema EMP_PROJ in Figure; from the semantics of the attributes and the relation, the following functional dependencies should hold: $Ssn \rightarrow Ename$

$Pnumber \rightarrow \{Pname, Plocation\}$

$\{Ssn, Pnumber\} \rightarrow Hours$

These functional dependencies specify that

- The value of an employee's Social Security number (Ssn) uniquely determines the employee name (Ename),
- The value of a project's number (Pnumber) uniquely determines the project name (Pname) and location (Plocation),
- A combination of Ssn and Pnumber values uniquely determines the number of hours the employee currently works on the project per week (Hours).
- Alternatively, Ename is functionally determined by (or functionally dependent on) Ssn.

2.5 NON-LOSS(LOSSLESS-JOIN) DECOMPOSITION

Let R be a relation schema, and let F be a set of functional dependencies on R. Let R1 and R2 form a decomposition of R.

This decomposition is a lossless-join decomposition of R if at least one of the following functional dependencies is in F+:

$$R1 \cap R2 \rightarrow R1 \text{ or } R1 \cap R2 \rightarrow R2$$

In other words, if $R1 \cap R2$ forms a superkey of either R1 or R2, the decomposition of R is a lossless-join decomposition. Example

1:

- Consider following lending relation:

Lending_Schema = (Branch_name, Branch_city, Assets, Customer_name, Loan_No, Amount)

- The decomposition of lending relation given below is lossless-join decomposition. It is described below:

➤ We now decompose the lending relation into two relations:

1. Branch_schema = (Branch_name, Branch_city, Assets)

2. Loan_info_schema = (Branch_name, Customer_name, Loan_no, Amount)

➤ Branch_schema $\cap$ Loan_info_schema is *branch_name* which is common attribute and

Branch_name $\rightarrow$ Branch_city, Assets. (By augmentation rule) Branch_name $\rightarrow$ Branch_name, Branch_city, Assets

Which is nothing but a *Branch_schema* relation. Thus the first clause is satisfied.

Hence, the decomposition of lending relation into:

Branch_schema and *Loan_info_schema* is lossless-join decomposition.

➤ Next, we decompose the Loan_info_schema into: **Loan_schema = (Loan_No, Branch_name, Amount)** **Borrower_schema =**

(Customer_name, Loan_No)

- This decomposition is also lossless-join decomposition since Loan_No is a common attribute and $Loan_No \twoheadrightarrow Amount, Branch_name$.

2.6 Normal Forms Based on Primary Keys

2.6.1 Normalization of Relations:

The normalization process, as first proposed by Codd (1972a), takes a relation schema through a series of tests to *certify* whether it satisfies a certain **normal form**.

2.6.2 Normalization of data:

1. It can be considered a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of (1) minimizing redundancy and (2) minimizing the insertion, deletion, and update anomalies.

2. Unsatisfactory relation schemas that do not meet certain conditions—the **normal form tests** are decomposed into smaller relation schemas that meet the tests and hence possess the desirable properties.

3. **Definition:** The **normal form** of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.

4. Normalization must confirm the existence of additional properties:

5. The **nonadditive join or lossless join property**, which guarantees that the spurious tuple generation problem does not occur with respect to the relation schemas created after decomposition.

6. The **dependency preservation property**, which ensures that each functional dependency is represented in some individual relation resulting after decomposition.

2.6.3 Denormalization:

It is the process of storing the join of higher normal form relations as a base relation, which is in a lower normal form.

2.6.4 Definitions of Keys and Attributes Participating in Keys

1. A **key** K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey any more.

2. If a relation schema has more than one key, each is called a **candidate key**.

3. One of the candidate keys is arbitrarily designated to be the **primary key**, and the others are called secondary keys.

4. An attribute of relation schema R is called a **prime attribute** of R if it is a member of some candidate key of R .

5. An attribute is called **nonprime** if it is not a prime attribute, that is, if it is not a member of any candidate key.

2.6.5 First Normal form

- It states that the domain of an attribute must include only **atomic** (simple, indivisible) **values** and that the value of any attribute in a tuple must be a **single value** from the domain of that attribute.
- It disallows having a set of values, a tuple of values, or a combination of both as an attribute value for a single tuple.

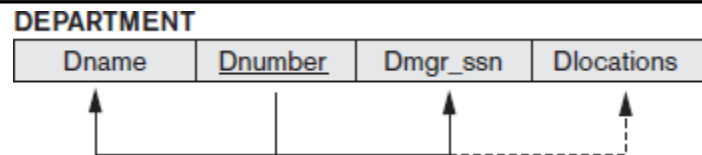


Fig 2.8 . A relation schema that is not in 1NF

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

Fig2.9 Sample state of relation DEPARTMENT

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Fig 2.10 1NF version of the same relation with redundancy

- Fig is not in 1NF because Dlocations is not an atomic attribute.
- There are three main techniques to achieve first normal form:

First technique:

1. Remove the attribute Dlocations and place it in a separate relation DEPT_LOCATIONS, along with the primary key Dnumber.
2. The primary key of this relation is the combination {Dnumber, Dlocation}.
3. A distinct tuple in DEPT_LOCATIONS exists for *each location* of a department.
4. This decomposes the non-1NF relation into two 1NF relations.

Second Technique:

1. Expand the key so that there will be a separate tuple, in the original DEPARTMENT relation for each location of a DEPARTMENT.
2. The primary key becomes the combination {Dnumber, Dlocation}.

Disadvantage: introducing *redundancy* in the relation.

Third technique:

1. If a *maximum number of values* is known for the attribute—for example, if it is known that *at most three locations* can exist for a department—replace the Dlocations attribute by three atomic attributes: Dlocation1, Dlocation2, and Dlocation3.

Disadvantage: Introducing *NULL values* if most departments have fewer than three locations.

The first solution is considered best because it does not suffer from redundancy and it is completely general, having no limit placed on a maximum number of values.

2.6.6 Second Normal Form

1. It is based on the concept of **full functional dependency**.
2. A functional dependency $X \rightarrow Y$ is a **full functional dependency** if removal of any attribute A from X means that the dependency does not hold any more.
3. A functional dependency $X \rightarrow Y$ is a **partial dependency** if some attribute $A \in X$ can be removed from X and the dependency still holds.
4. In the following figure, $\{Ssn, Pnumber\} \rightarrow Hours$ is a full dependency (neither $Ssn \rightarrow Hours$ nor $Pnumber \rightarrow Hours$ holds).
5. However, the dependency $\{Ssn, Pnumber\} \rightarrow Ename$ is partial because $Ssn \rightarrow Ename$ holds.

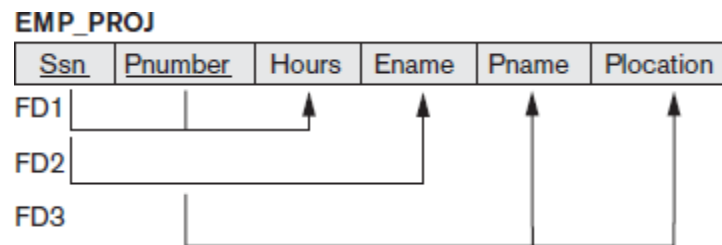


Fig2.11 Relation schema EMP_PROJ

6. The EMP_PROJ relation is in 1NF but is not in 2NF.
7. The functional dependencies FD2 and FD3 make Ename, Pname, and Plocation partially dependent on the primary key $\{Ssn, Pnumber\}$ of EMP_PROJ.
8. If a relation schema is not in 2NF, it can be **second normalized or 2NF normalized** into a number of 2NF relations.

9. In that 2NF Relation , nonprime attributes are associated only with the part of the primary key on which they are fully functionally dependent.
10. The functional dependencies FD1, FD2, and FD3 lead to the decomposition of EMP_PROJ into the three relation schemas EP1, EP2, and EP3 shown in figure, each of which is in 2NF.

2NF Normalization

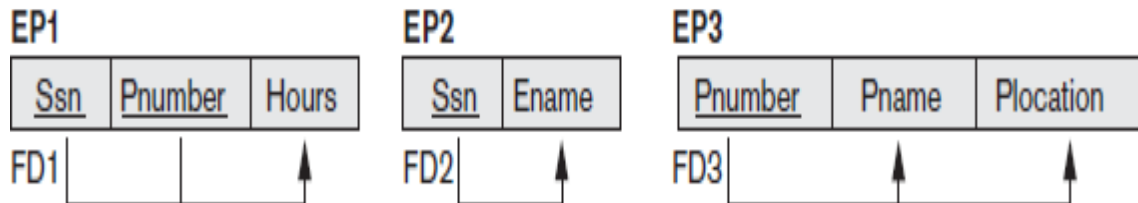


Fig2.12 Normalizing EMP_PROJ into 2NF relations

2.6.7 Third Normal Form

1. It is based on the concept of transitive dependency.
2. A functional dependency $X \rightarrow Y$ in a relation schema R is a **transitive dependency** if there exists a set of attributes Z in R that is neither a candidate key nor a subset of any key of R , and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.
3. The dependency $Ssn \rightarrow Dmgr_ssn$ is transitive through $Dnumber$ in EMP_DEPT in figure, because both the dependencies $Ssn \rightarrow Dnumber$ and $Dnumber \rightarrow Dmgr_ssn$ hold and $Dnumber$ is neither a key itself nor a subset of the key of EMP_DEPT.

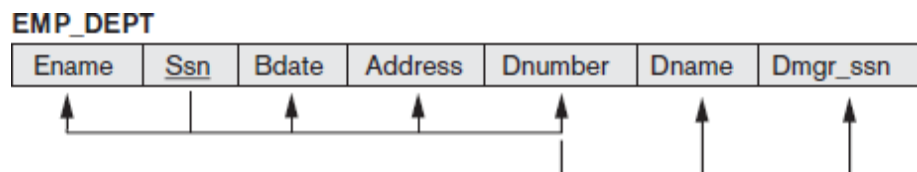


Fig2.13. Relation schema EMP_DEPT

Definition: A relation schema R is in **3NF** if it satisfies 2NF *and* no nonprime attribute of R is transitively dependent on the primary key.

- The relation schema EMP_DEPT is in 2NF but not in 3NF because of the transitive dependency.

- EMP_DEPT is normalized by decomposing it into the two 3NF relation schemas ED1 and ED2.

3NF Normalization

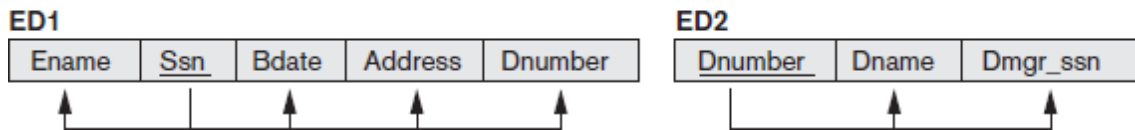


Fig2.14. Normalizing EMP_DEPT into 3NF relations

2.7 BOYCE CODD NORMAL FORM

Definition: A relation schema R is in **BCNF** if whenever a *nontrivial* functional dependency $X \rightarrow A$ holds in R , then X is a superkey of R .

1. Example: Consider a relation TEACH with the following dependencies:

FD1: {Student, Course} $\rightarrow$ Instructor FD2:

Instructor $\rightarrow$ Course

2. {Student, Course} is a candidate key for this relation and that the dependencies shown follow the pattern in figure, with Student as A , Course as B , and Instructor as C .

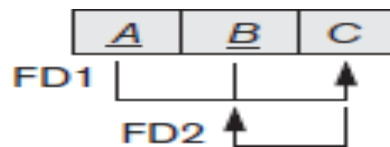


Fig2.15. A schematic relation with FDs; it is in 3NF, but not in BCNF

3. Hence this relation is in 3NF but not BCNF.

4. Decomposition of this relation schema into two schemas is not straightforward because it may be decomposed into one of the three following possible pairs:

{Student, Instructor} and {Student, Course}

{Course, Instructor} and {Course, Student}

{Instructor, Course} and {Instructor, Student}

5. All three decompositions *lose* the functional dependency FD1. The *desirable decomposition* of those just shown is 3 because it will not generate spurious tuples after a join.

6. A relation not in BCNF should be decomposed so as to meet this property. Nonadditive decomposition is a must during normalization.

2.8 FORMAL DEFINITION OF MULTIVALUED DEPENDENCIES(MVD):

The MVD $x \twoheadrightarrow Y$ is said to hold for $R(X,Y,Z)$ if, whenever t_1 and t_2 are two rows in R that have the same values for attributes X and therefore $t_1[x]=t_2[x]$ then R also contains t_3 and t_4 , such that

$$t_3[X] = t_4[X] = t_1[X] = t_2[X]$$

$$t_3[Y] = t_1[Y] \text{ and } t_4[Y] = t_2[Y]$$

$$t_3[Z] = t_2[Z] \text{ and } t_4[Z] = t_1[Z]$$

2.8.1 Fourth Normal Form

A relation schema R is in 4NF with respect to a set of dependencies F if, for every *nontrivial* multivalued dependency $X \twoheadrightarrow Y$ in F^+ , X is a superkey for R .

Consider the EMP relation in figure. EMP is not in 4NF because in the nontrivial MVDs $\text{Ename} \twoheadrightarrow \text{Pname}$ and $\text{Ename} \twoheadrightarrow \text{Dname}$, and Ename is not a superkey of EMP.

EMP

<u>Ename</u>	<u>Pname</u>	<u>Dname</u>
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

Fig2.16. The EMP relation with two MVDs: $\text{Ename} \twoheadrightarrow \text{Pname}$ and $\text{Ename} \twoheadrightarrow \text{Dname}$

Decompose EMP into EMP_PROJECTS and EMP_DEPENDENTS, shown in figure.

Both EMP_PROJECTS and EMP_DEPENDENTS are in 4NF, because the MVDs $\text{Ename} \twoheadrightarrow \text{Pname}$ in EMP_PROJECTS and $\text{Ename} \twoheadrightarrow \text{Dname}$ in EMP_DEPENDENTS are trivial MVDs.

No other nontrivial MVDs hold in either EMP_PROJECTS or EMP_DEPENDENTS. No FDs hold in these relation schemas either.

EMP_PROJECTS

<u>Ename</u>	<u>Pname</u>
Smith	X
Smith	Y

EMP_DEPENDENTS

<u>Ename</u>	<u>Dname</u>
Smith	John
Smith	Anna

Fig2.17. Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS

2.9 JOIN DEPENDENCIES

Let a relation R have subset of its attribute A,B,C ,..Then R satisfies the Join dependency (JD) written as $\bowtie(A,B,C)$ if and only if every possible legal value of R is equal to the join of its projection A,B,C...

2.9.1 Definition of 5NF:

A relation R is in 5NF (or project-join normal form, PJNF) if for all join dependencies of the form $\bowtie(R_1, R_2, \dots, R_n)$, where each R_i is a subset of the set of attributes of R and $R = R_1 R_2 \dots R_n$, at least one of the following holds.

$\bowtie(R_1, R_2, \dots, R_n)$ is a trivial join-dependency (i.e., one of R_i is R)

$\forall$ Every R_i is a super key for R.

Example:

Department	Subject	Student
Comp. Sc.	CP1000	John Smith
Mathematics	MA1000	John Smith
Comp. Sc.	CP2000	Arun Kumar
Comp. Sc.	CP3000	Reena Rani
Physics	PH1000	Raymond Chew
Chemistry	CH2000	Albert Garcia

1. The above relation says that Comp. Sc. offers subjects CP1000, CP2000 and CP3000 which are taken by a variety of students.

2. No student takes all the subjects and no subject has all students enrolled in it and therefore all three fields are needed to represent the information.

3. The above relation does not show MVDs since the attributes subject and student are not independent; they are related to each other and the pairings have significant information in them.

4. The relation can therefore not be decomposed in two relations (dept, subject), and (dept, student)

Without losing some important information.

The relation can however be decomposed in the following three relations (dept, subject), and

(dept, student)

(subject, student)

Now it can be shown that this decomposition is lossless.